

```

import java.util.Iterator;

import components.sequence.Sequence;
import components.sequence.SequenceSecondary;
import components.stack.Stack;
import components.stack.Stack1L;

/**
 * {@code Sequence} represented as a pair of {@code Stack}s with implementations
 * of primary methods.
 *
 * @param <T>
 *         type of {@code Sequence} entries
 * @correspondence this = rev($this.left) * $this.right
 */
public class Sequence3<T> extends SequenceSecondary<T> {

    /**
     * Private members -----
     */

    /**
     * Left stack.
     */
    private Stack<T> left;

    /**
     * Right stack.
     */
    private Stack<T> right;

    /**
     * Shifts entries between {@code leftStack} and {@code rightStack}, keeping
     * reverse of the former concatenated with the latter fixed, and resulting
     * in length of the former equal to {@code newLeftLength}.
     *
     * @param <T>
     *         type of {@code Stack} entries
     * @param leftStack
     *         the left {@code Stack}
     * @param rightStack
     *         the right {@code Stack}
     * @param newLeftLength
     *         desired new length of {@code leftStack}
     * @updates leftStack, rightStack
     * @requires <pre>
     * 0 <= newLeftLength and
     * newLeftLength <= |leftStack| + |rightStack|
     * </pre>
     * @ensures <pre>
     * rev(leftStack) * rightStack = rev(#leftStack) * #rightStack and
     * |leftStack| = newLeftLength
     * </pre>
     */
    private static <T> void setLengthOfLeftStack(Stack<T> leftStack,
        Stack<T> rightStack, int newLeftLength) {
        assert leftStack != null : "Violation of: rightStack is not null";
        assert leftStack != null : "Violation of: rightStack is not null";
        assert 0 <= newLeftLength : "Violation of: 0 <= newLeftLength";
        assert newLeftLength <= leftStack.length() + rightStack.length() : ""
            + "Violation of: newLeftLength <= |leftStack| + |rightStack|";

        while (leftStack.length() < newLeftLength) {
            T x = rightStack.pop();
            leftStack.push(x);
        }

        while (leftStack.length() > newLeftLength) {
            T x = leftStack.pop();

```

```

        rightStack.push(x);
    }
}

/**
 * Creator of initial representation.
 */
private void createNewRep() {
    this.left = new Stack1L<T>();
    this.right = new Stack1L<T>();
}

/*
 * Constructors -----
 */

/**
 * No-argument constructor.
 */
public Sequence3() {
    this.createNewRep();
}

/*
 * Standard methods -----
 */

@SuppressWarnings("unchecked")
@Override
public final Sequence<T> newInstance() {
    try {
        return this.getClass().getConstructor().newInstance();
    } catch (ReflectiveOperationException e) {
        throw new AssertionError(
            "Cannot construct object of type " + this.getClass());
    }
}

@Override
public final void clear() {
    this.createNewRep();
}

@Override
public final void transferFrom(Sequence<T> source) {
    assert source != null : "Violation of: source is not null";
    assert source != this : "Violation of: source is not this";
    assert source instanceof Sequence3<?> : ""
        + "Violation of: source is of dynamic type Sequence3<?>";
    /*
     * This cast cannot fail since the assert above would have stopped
     * execution in that case: source must be of dynamic type Sequence3<?>,
     * and the ? must be T or the call would not have compiled.
     */
    Sequence3<T> localSource = (Sequence3<T>) source;
    this.left = localSource.left;
    this.right = localSource.right;
    localSource.createNewRep();
}

/*
 * Kernel methods -----
 */

@Override
public final void add(int pos, T x) {
    assert x != null : "Violation of: x is not null";
    assert 0 <= pos : "Violation of: 0 <= pos";
    assert pos <= this.length() : "Violation of: pos <= |this|";
}

```

```
        // TODO - fill in body
    }

    @Override
    public final T remove(int pos) {
        assert 0 <= pos : "Violation of: 0 <= pos";
        assert pos < this.length() : "Violation of: pos < |this|";

        // TODO - fill in body

        // This line added just to make the component compilable.
        return null;
    }

    @Override
    public final int length() {

        // TODO - fill in body

        // This line added just to make the component compilable.
        return 0;
    }

    @Override
    public final Iterator<T> iterator() {
        setLengthOfLeftStack(this.left, this.right, 0);
        return this.right.iterator();
    }
}
```